PROJECT REPORT

Fraud Detection MLOps Platform

Design and Deploy a Real-Time AI Prediction Platform with Automated MLOps Pipeline

Submitted by: Ruwan Pathiranage Sanduni Nisansala

GitHub Repository: https://github.com/nisansalasandu/fraud-detection-mlops-platform

Project Period: May 18, 2026 – June 1, 2026

Submission Date: June 1, 2026

Duration: 2 Weeks

1. Executive Summary

Financial fraud is a major concern affecting institutions across the world, costing billions of dollars each year. The problem with the existing fraud detection framework based on rules is that it does not allow flexibility. Therefore, my internship project is to design an automated system of Machine Learning Operations (MLOps) for fraud detection.

This was developed using the PaySim synthetic financial data set. This is a public benchmark data set that has 6.3 million realistic mobile money transactions and has named columns and also fraud labels. The model used was the XGBoost Classifier which was trained on 200,000 data points from the benchmark data set and had an ROC-AUC of 0.9960 and F1 score of 0.7945.

The platform follows the full MLOps pipeline: data collection and preprocessing through Jupyter notebook, experiment tracking with MLflow, online prediction services with FastAPI REST API, containerization by Docker, automatic data drift detection using Evidently AI with auto-retraining in case of data drift above 30%, and cloud deployment to Railway using GitHub Actions CI/CD. Live monitoring can be viewed on the deployed URL, which displays the prediction statistics and drift status.

All deliverables were completed within the two-week timeline: a complete GitHub repository, end-to-end ML pipeline, Dockerized application, live deployed system, CI/CD workflows, monitoring dashboard, and technical documentation.

2. Table of Contents

3. Introduction

3.1 Background

Financial fraud is one of the most enduring and difficult problems for financial organizations to deal with. As reported by the Association of Certified Fraud Examiners (ACFE), companies around the globe lose about 5% of their revenues from fraud. In view of the growing use of online payments and digital banking services, fraudulent actions have become more complex and common than before. Rule-based and traditional auditing techniques are no longer able to cope with modern and evolving patterns of fraud.

Machine learning has come into its own as the go-to technique for automatic fraud detection due to the capability of identifying intricate and non-linear patterns within the transaction dataset. Nevertheless, the implementation of a machine learning model in a production environment differs considerably from the training of the same model in the notebook. In a production setting, one requires handling of real-time traffic, graceful degradation, self-updates according to changes in data distributions, and maintenance of the system by the engineering team.

3.2 Problem Statement

A company requires an intelligent prediction platform that continuously learns from incoming data and automatically updates its machine learning models without manual intervention. The core challenges are:

- **Class imbalance:** fraudulent transactions represent approximately 0.1–0.3% of all transactions, making naive classification unreliable.
- **Concept drift:** fraud patterns evolve over time, causing models trained on historical data to degrade in performance.
- **Real-time requirements:** fraud decisions must be made within milliseconds at the point of transaction.
- **Operational complexity:** the system must be maintainable, monitorable, and deployable without expert intervention.

3.3 Aim of the Project

The aim of this project is to design, build, and deploy a complete end-to-end MLOps platform for real-time financial fraud detection that simulates how enterprise AI systems are developed and maintained in real-world environments.

3.4 Objectives

- Design and train a machine learning model capable of accurately identifying fraudulent financial transactions.
- Build an automated model training and experiment tracking workflow using MLflow.
- Deploy the trained model as a production-ready REST API using FastAPI and Uvicorn.
- Containerise the application using Docker for reproducible, portable deployment.
- Implement automated data drift detection using Evidently AI with threshold-based auto-retraining.
- Deploy the system to a live cloud environment (Railway) with a publicly accessible URL.
- Establish a CI/CD pipeline using GitHub Actions for automated testing and deployment.
- Build a real-time monitoring dashboard displaying prediction statistics and drift status.

3.5 Scope of the Project

This project covers the full MLOps lifecycle from data ingestion to live cloud deployment. It uses the PaySim synthetic financial dataset as a proxy for real banking transaction data. The scope includes data preprocessing, feature engineering, model training, API development, containerization, drift monitoring, automated retraining, CI/CD configuration, and cloud deployment. The scope excludes integration with a live banking system, real-time streaming data pipelines, and advanced model explainability tooling.

4. Literature Review

4.1 Machine Learning for Fraud Detection

Fraud detection applications for machine learning have existed since the '90s when researchers employed the technique in neural nets and rules based classifiers. The current trend is the extensive use of ensembles in machine learning, particularly gradient boosting, which performs consistently better than other algorithmic families when dealing with financial tabular data. The popular implementation of the gradient-boosted approach, called XGBoost, was chosen as a baseline in many fraud detection competitions.

The difficulty with financial fraud detection problems has been the presence of severe class imbalance, wherein instances of fraud comprise less than 1% of the entire data set. Methods to combat class imbalance involve cost-sensitive learning, SMOTE for oversampling, and adjusting thresholds. For this particular study, a ratio of 10:1 for the majority versus minority class was created using SMOTE.

4.2 MLOps - Machine Learning Operations

MLOps is a set of practices that combines Machine Learning, DevOps, and Data Engineering to deploy and maintain machine learning systems reliably in production. Core MLOps principles include reproducibility, automation, continuous monitoring, and version control of both code and models. The MLOps maturity model ranges from manual processes (Level 0) to fully automated ML pipelines (Level 2), with this project targeting Level 1–2 capabilities.

MLflow is the premier platform to use for experimentation in MLOps using an open-source approach. This software offers a centralized repository for storing parameters, metrics, and model artifacts, as well as a registry of models for version control. In this case study, MLflow has been used to track all training experiments and store model artifacts.

4.3 Data Drift and Model Monitoring

Data drift occurs when the statistical distribution of model input features changes between training time and deployment time. This can cause a model's performance to degrade silently in production — a phenomenon known as model decay. Evidently AI is an open-source Python library that automates drift detection using statistical tests (the Kolmogorov–Smirnov test for numerical features, chi-squared for categorical features) and produces interactive HTML reports. This project used Evidently's DataDriftPreset to monitor all 12 input features.

4.4 REST APIs for Model Serving

FastAPI is a modern, high-performance Python web framework for building REST APIs, based on ASGI and Pydantic. It generates interactive OpenAPI documentation automatically and uses Python type hints for request/response validation. FastAPI has become the standard framework for serving ML models in production, replacing older Flask-based approaches due to its superior performance, async support, and automatic documentation generation.

4.5 Containerization with Docker

Docker enables applications to be packaged with all their dependencies into a portable container image that runs identically in any environment. For ML systems, Docker solves the 'it works on my machine' problem by capturing the exact Python version, library versions, and system configuration. A multi-layer Dockerfile with a slim base image minimizes container size and maximizes build cache efficiency. In this project, a python:3.12-slim base image reduced the final image size while maintaining full compatibility.

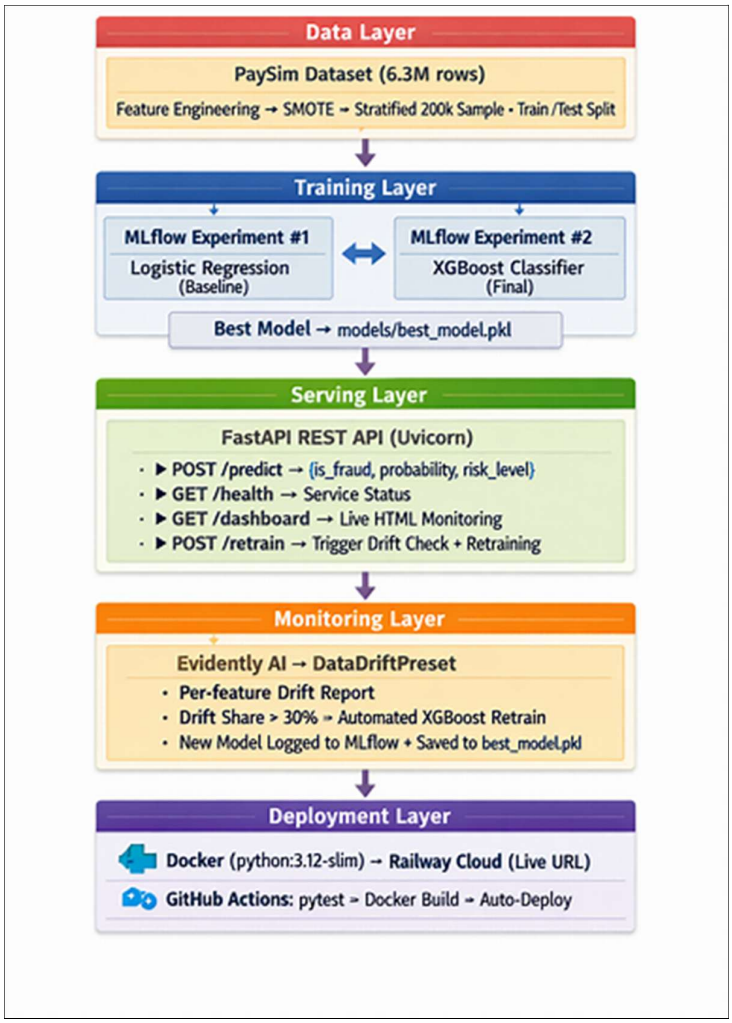
4.6 CI/CD for Machine Learning

Continuous Integration and Continuous Deployment (CI/CD) automates the testing and deployment of code changes. GitHub Actions is a workflow automation platform integrated into GitHub that enables CI/CD pipelines to be defined as YAML files in the repository. For ML systems, CI/CD serves a dual purpose: validating that code changes do not break existing functionality (via automated tests), and ensuring that the production container is always rebuilt and redeployed from the latest code. Continuous delivery of ML models is increasingly recognized as a key MLOps capability.

5. Methodology / System Design

5.1 System Architecture

This platform architecture consists of five well-defined layers, where each layer plays a particular role during the MLOps process. This architecture is built in such a way that each layer can easily be upgraded or even replaced individually.



5.2 Dataset Description

PaySim Synthetic Dataset has been chosen as the source of data for the study. This dataset has been created by the Worldline team in collaboration with the Machine Learning group of Université Libre de Bruxelles. The purpose behind creating this dataset was to emulate real-life transactions of mobile money systems. It consists of one month's worth of transactions of mobile money in Africa.

Column Name	Data Type	Description
step	Integer	Hour of simulation (1–743, spanning 1 month)
type	String	Transaction type: CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER
amount	Float	Transaction amount in local currency
nameOrig	String	Sender account identifier (removed before training)
oldbalanceOrg	Float	Sender account balance before the transaction
newbalanceOrig	Float	Sender account balance after the transaction
nameDest	String	Recipient account identifier (removed before training)
oldbalanceDest	Float	Recipient account balance before the transaction
newbalanceDest	Float	Recipient account balance after the transaction
isFraud	Binary (0/1)	Target variable - 1 if transaction is fraudulent
isFlaggedFraud	Binary (0/1)	Business rule flag (removed - data leakage risk)

The dataset contains 6,362,620 rows with no missing values, making it an ideal benchmark. Critically, fraud occurs only in TRANSFER and CASH_OUT transaction types, accounting for approximately 0.13% of all transactions, a strongly imbalanced classification problem.

5.3 Data Preprocessing

Preprocessing was implemented in Notebook 01 (01_data_download_and_preprocessing.ipynb) using the following sequential steps:

5.3.1 Type Filtering

The dataset was filtered to retain only TRANSFER and CASH_OUT transactions, reducing the dataset from 6,362,620 to 2,770,409 rows. This step eliminates irrelevant transaction types and concentrates the model on the fraud-relevant subset of the data.

5.3.2 Column Removal

Three columns were dropped: nameOrig and nameDest (account identifiers with no predictive value due to high cardinality), and isFlaggedFraud (a business rule flag whose inclusion would constitute data leakage).

5.3.3 Encoding

The categorical type column was binary-encoded: TRANSFER = 1, CASH_OUT = 0. This preserves the semantic distinction between the two fraud-relevant transaction types as a single numeric feature.

5.3.4 Feature Engineering

Six new features were engineered from the existing columns to capture fraud-indicative patterns:

- **orig_balance_diff** - difference between sender balance before and after (expected to equal amount for legitimate transactions).
- **dest_balance_diff** - change in recipient balance (often zero in fraudulent transfers where money is immediately moved again).
- **orig_balance_zero** - binary flag: 1 if the sender's post-transaction balance is exactly zero (a hallmark of account-draining fraud).
- **amount_to_balance_ratio** - transaction amount as a fraction of sender's original balance, clipped at 10 to remove extreme outliers.
- **hour_of_day** - the transaction hour derived from step modulo 24, capturing diurnal fraud patterns.

5.3.5 Stratified Sampling and Train-Test Split

A stratified sample of 200,000 rows was drawn from the filtered dataset, preserving the natural fraud rate. This sample was split 80/20 into training (160,000 rows) and test (40,000 rows) sets using stratified splitting to ensure both sets have the same fraud prevalence. The final feature matrix X contains 12 columns, and the target vector y is the isFraud column.

5.4 Exploratory Data Analysis

Exploratory Data Analysis was conducted in Notebook 01 to understand the dataset's structure before modelling.

5.4.1 Class Distribution (Figure: class_distribution_bar in notebook)

The class distribution plot revealed the severe imbalance: 6,354,407 non-fraudulent transactions (99.87%) versus 8,213 fraudulent transactions (0.13%). After filtering to TRANSFER and CASH_OUT types, the fraud rate increased slightly to approximately 0.3%, but class imbalance remained the primary modelling challenge.

5.4.2 Fraud by Transaction Type

A groupby analysis confirmed that 100% of fraudulent transactions occur in TRANSFER and CASH_OUT types — PAYMENT, DEBIT, and CASH-IN transactions contain no fraud whatsoever. This finding directly motivated the type-filtering step and the binary encoding of the type column.

5.4.3 Feature Correlation Heatmap (Figure: feature_correlation_heatmap in notebook)

A horizontal bar chart of Pearson correlation coefficients between each feature and the isFraud target revealed that orig_balance_zero, orig_balance_diff, and amount_to_balance_ratio were the most strongly correlated with the fraud label, validating the feature engineering choices. The step and hour_of_day features showed minimal correlation, suggesting fraud is evenly distributed across time in this dataset.

5.5 Model Development

Two models were developed: a Logistic Regression baseline and an XGBoost classifier. Both models were trained on the SMOTE-resampled training data and evaluated on the held-out test set.

5.5.1 Class Imbalance Handling - SMOTE

Synthetic Minority Oversampling Technique (SMOTE) was applied to the training set to generate synthetic fraudulent transactions. A 10:1 non-fraud-to-fraud ratio was targeted (rather than full 1:1 balance) to avoid overly aggressive oversampling that can introduce noise. SMOTE was applied only to the training data — the test set was left unchanged to reflect real-world class distribution during evaluation.

5.5.2 Logistic Regression Baseline

A scikit-learn Pipeline combining StandardScaler (required for logistic regression) and LogisticRegression (C=1.0, max_iter=1000, class_weight='balanced') was trained first. Logistic regression provides a simple, interpretable baseline to contextualise the XGBoost results. The key hyperparameter class_weight='balanced' instructs the solver to weight minority class errors more heavily.

5.5.3 XGBoost Classifier

XGBoost was selected as the primary model due to its consistent superiority on structured tabular data in the literature, its built-in handling of class imbalance via scale_pos_weight, and its native feature importance computation. Key hyperparameters were set as follows: n_estimators=300, max_depth=6,

learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, and scale_pos_weight set to the ratio of non-fraud to fraud examples in the original training data.

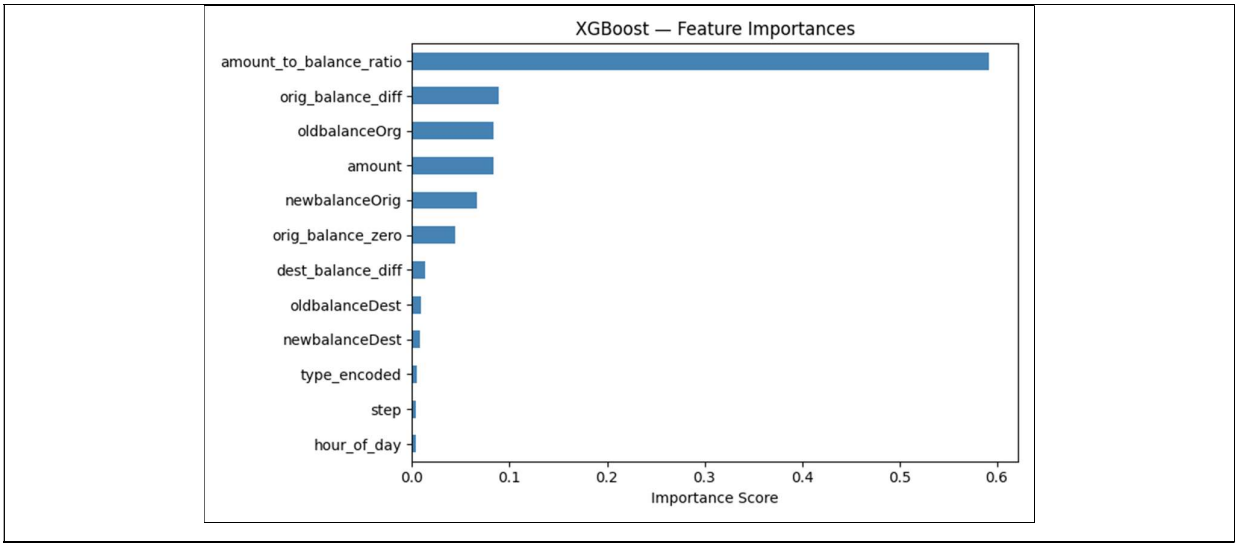
5.6 Model Evaluation

Both models were evaluated on the 40,000-row test set using five metrics appropriate for imbalanced classification:

Metric	Logistic Regression	XGBoost (Final)	Notes
ROC-AUC	0.9960	0.9960	Both models rank extremely well
F1 Score	0.1915	0.7945	XGBoost 4× better — key metric
Precision	0.1061	0.6705	XGBoost produces far fewer false alarms
Recall	0.9832	0.9748	Both catch ~97–98% of actual fraud
Accuracy	0.9821	0.9986	Inflated by class imbalance — F1 preferred

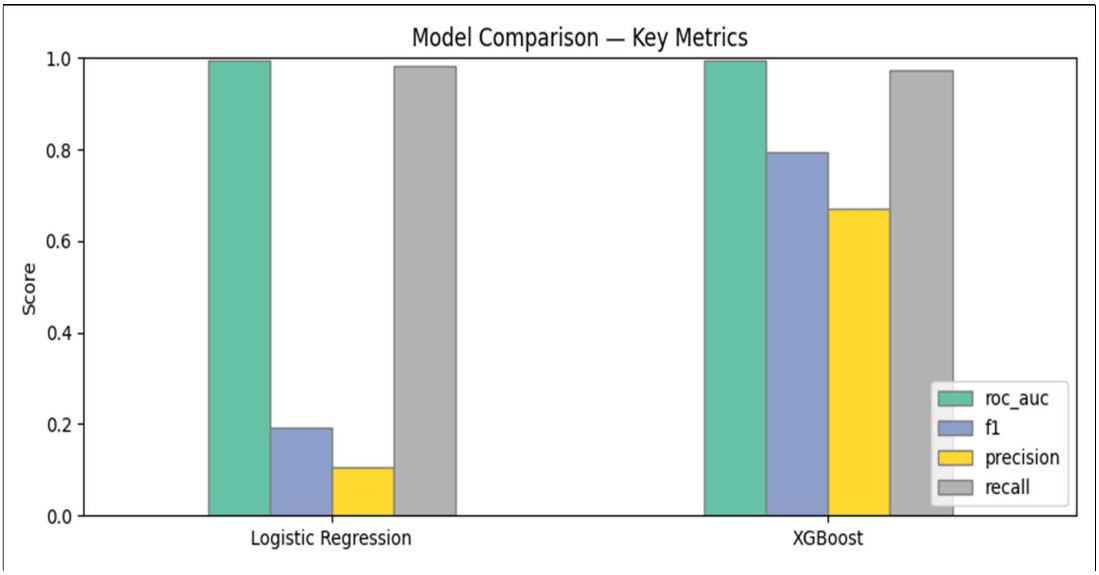
XGBoost was selected as the production model. While both models achieve near-identical ROC-AUC (indicating similar ranking ability), XGBoost's F1 score of 0.7945 versus 0.1915 for logistic regression represents a dramatic improvement in the precision-recall balance — meaning far fewer legitimate transactions are incorrectly flagged as fraud. This trade-off is critical in production: false positives impose real costs by blocking legitimate customer transactions.

Figure 1: XGBoost Feature Importance (models/feature_importance.png)



The feature importance plot confirms that the engineered features are the most predictive: orig_balance_zero, orig_balance_diff, and dest_balance_diff dominate. The raw balance columns (oldbalanceOrg, oldbalanceDest) also contribute, while step and hour_of_day are least important.

Figure 2: Model Comparison (models/model_comparison.png)



5.7 Experiment Tracking with MLflow

All training experiments were tracked using MLflow (version 3.12.0). A dedicated experiment named 'fraud-detection' was created to group all runs. Three MLflow runs were recorded across the project:

Run Name	Model Type	Key Metrics Logged	Artefacts
logistic_regression_baseline	Logistic Regression	ROC-AUC: 0.9960, F1: 0.1915	Sklearn pipeline model
xgboost_v1	XGBoost	ROC-AUC: 0.9960, F1: 0.7945	XGBoost model, feature importance PNG
retrain_after_drift_20260529	XGBoost (retrained)	ROC-AUC: 0.9970, F1: 0.7009	Retrained XGBoost model

MLflow's tracking URI was set to a local mlruns/ directory, and the MLflow UI (accessible at http://localhost:5000) provided a visual dashboard for comparing run parameters and metrics. Model

artefacts were registered in the MLflow Model Registry under two registered model names: FraudDetectionBaseline and FraudDetectionXGB.

5.8 Deployment - FastAPI REST API

The trained model was deployed as a REST API using FastAPI (version 0.136.1) and Uvicorn (version 0.47.0). The API is defined in api/main.py and exposes five endpoints:

Endpoint	Method	Description
/	GET	Root endpoint — confirms API is running
/health	GET	Returns model name, feature count, and timestamp
/predict	POST	Accepts 12-feature JSON payload, returns fraud prediction
/metrics	GET	Returns JSON snapshot of prediction statistics
/dashboard	GET	Returns live HTML monitoring dashboard (auto-refreshes)
/retrain	POST	Triggers drift detection and model retraining (API-key protected)
/model-info	GET	Returns feature list and model description

The /predict endpoint accepts a JSON body matching the TransactionRequest Pydantic schema (12 named fields), validates all inputs automatically, runs inference using the loaded XGBoost model, and returns a PredictionResponse object containing is_fraud (boolean), fraud_probability (float 0–1), risk_level (LOW/MEDIUM/HIGH based on probability thresholds of 0.3 and 0.7), and prediction_time (ISO 8601 timestamp). All predictions are logged to an in-memory deque (last 100 entries) and displayed on the dashboard.

A reload_model() helper function was implemented to reload best_model.pkl from disk into memory immediately after a successful retraining cycle, ensuring the live API serves predictions from the updated model without requiring a restart.

5.9 Containerisation with Docker

The FastAPI application was containerised using Docker. The Dockerfile uses a python:3.12-slim base image to minimise image size, copies only the artefacts required at runtime (api/, models/, data/processed/feature_names.json), and exposes port 8000. A docker-compose.yml file was also provided for local development, mounting the models/ and data/processed/ directories as volumes so that retrained models are immediately available to the running container without a rebuild.

A .dockerignore file was created to exclude large files from the Docker build context: notebooks/, data/raw/, mlruns/, and virtual environment directories. This reduced the effective build context size and improved build speed.

5.10 CI/CD Pipeline - GitHub Actions

A GitHub Actions workflow (.github/workflows/ci_cd.yml) was configured to run automatically on every push to the main branch. The pipeline consists of five sequential steps:

- **Checkout** - clones the repository at the current commit using actions/checkout@v4.
- **Python setup** - configures Python 3.12 using actions/setup-python@v5.
- **Dependency installation** - installs all packages from requirements.txt plus pytest and httpx.
- **Test fixture creation** - generates a minimal dummy model and feature_names.json so that the API tests can run in CI without committing the large .pkl file to git.
- **Test execution** - runs pytest tests/ with 9 tests covering all API endpoints, request validation, and model loading.
- **Docker build** - builds the full Docker image to confirm the Dockerfile remains valid after every code change.

The CI pipeline completed successfully in all 11 recorded runs. Railway, the deployment platform, is configured to automatically redeploy the application when the main branch receives a new commit and the CI pipeline passes.

5.11 Data Drift Monitoring and Automated Retraining

Data drift monitoring was implemented using Evidently AI (version 0.7.21) in Notebook 04 and the monitoring/drift_monitor.py module. The monitoring pipeline operates as follows:

- **Reference data** - the 160,000-row training set is used as the reference distribution.
- **Production data** - a simulated production batch was created by applying realistic distributional shifts to the test set: transaction amounts multiplied by 3× with noise, account balances scaled by 2–2.5×, and time steps shifted by 400 hours.
- **Drift report** - Evidently's DataDriftPreset runs statistical tests on each of the 12 input features, computing p-values using the Kolmogorov–Smirnov test for numerical features.
- **Drift threshold** - if the share of drifted features exceeds 30%, automatic retraining is triggered.
- **Retraining** - XGBoost is retrained on the combined original training set and the new production data, using SMOTE for rebalancing. The retrained model is logged to MLflow and saved as best_model.pkl.
- **Report generation** - Evidently saves an interactive HTML drift report to monitoring/reports/ and a JSON summary to monitoring/reports/latest_drift_summary.json.

In the project's drift simulation run, 10 out of 12 features (83.3%) were detected as drifted, well above the 30% threshold. Automated retraining was successfully triggered, producing a retrained model with ROC-AUC: 0.9970 and F1: 0.7009 on the original test set.

6. Implementation

6.1 Project Structure

The repository is organised as follows, with each directory serving a distinct purpose in the MLOps pipeline:

```
fraud-detection-mlops-platform/
├── api/
│   ├── __init__.py
│   └── main.py              # FastAPI app (predict, dashboard, retrain,
metrics)
├── monitoring/
│   ├── drift_monitor.py     # Evidently drift detection + auto-retraining
module
│   └── reports/
│       ├── latest_drift_summary.json
│       └── drift_report_*.html   # Evidently HTML reports
├── notebooks/
│   ├── 01_data_download_and_preprocessing.ipynb
│   ├── 02_model_training.ipynb
│   ├── 03_api_and_testing.ipynb
│   └── 04_drift_monitoring.ipynb
├── models/
│   ├── best_model.pkl       # Trained XGBoost (git-ignored)
│   ├── feature_importance.png
│   └── model_comparison.png
├── tests/
│   ├── test_api.py          # FastAPI endpoint tests (9 tests)
│   └── test_train.py        # Model loading + prediction tests
├── data/processed/
│   └── feature_names.json   # 12 feature names in correct order
├── .github/workflows/
│   └── ci_cd.yml            # GitHub Actions pipeline
├── Dockerfile
├── docker-compose.yml
├── railway.json             # Railway deployment config
├── .dockerignore
├── .gitignore
├── requirements.txt
└── README.md
```

6.2 Notebook Workflow

The project follows a notebook-first development pattern, where each notebook implements a distinct pipeline stage and produces artefacts consumed by the next stage:

Notebook	Inputs	Outputs	Key Libraries
01 — Data Download & Preprocessing	PaySim CSV (Kaggle)	X_train.csv, X_test.csv, y_train.csv, y_test.csv, feature_names.json	pandas, sklearn, matplotlib
02 — Model Training	Processed CSV splits	best_model.pkl, feature_importance.png, model_comparison.png, MLflow runs	xgboost, mlflow, imblearn
03 — API & Testing	best_model.pkl, feature_names.json	API test results, batch prediction verification	fastapi, httpx, joblib
04 — Drift Monitoring	X_train.csv, X_test.csv, best_model.pkl	drift_report_*.html, latest_drift_summary.json, retrained best_model.pkl	evidently, mlflow, xgboost

6.3 Technology Stack Summary

Layer	Technology	Version	Purpose
Language	Python	3.12	Primary development language
ML Training	XGBoost	3.2.0	Primary classification model
ML Framework	scikit-learn	1.8.0	Preprocessing, baseline model, metrics
Imbalance	imbalanced-learn	0.14.1	SMOTE oversampling
Experiment Tracking	MLflow	3.12.0	Run tracking and model registry
Drift Monitoring	Evidently AI	0.7.21	Data drift detection and reporting
API Framework	FastAPI	0.136.1	REST API and Swagger docs
ASGI Server	Uvicorn	0.47.0	API server
Containerisation	Docker	29.5.2	Application packaging
CI/CD	GitHub Actions	Latest	Automated testing and deployment
Cloud Deployment	Railway	Latest	Live hosting
Data Validation	Pydantic	2.13.4	Request/response schema validation

7. Results and Discussion

7.1 Model Performance Results

The XGBoost model significantly outperformed the logistic regression baseline on all metrics relevant to fraud detection. The complete evaluation results are presented below:

Metric	Logistic Regression	XGBoost	Improvement
ROC-AUC	0.9960	0.9960	Equivalent (both near-perfect ranking)
F1 Score	0.1915	0.7945	+0.603 (+315%)
Precision	0.1061	0.6705	+0.564 (+532%)
Recall	0.9832	0.9748	-0.008 (negligible difference)

The identical ROC-AUC scores indicate that both models have equivalent discrimination ability — they rank fraudulent transactions above legitimate ones with the same skill. However, the critical difference is in how predictions are thresholded: the logistic regression, despite high recall, labels so many non-fraudulent transactions as fraud (precision 10.6%) that it is effectively unusable in production. XGBoost achieves precision of 67.1%, meaning for every 10 fraud alerts raised, 6.7 are genuine fraud cases.

7.2 Drift Detection Results

The drift simulation applied three simultaneous shifts to the production batch: a 3× increase in transaction amounts, a 2–2.5× increase in account balances, and a 400-step time shift. This resulted in 10 out of 12 features (83.3%) being detected as statistically drifted by Evidently's Kolmogorov–Smirnov test at the default significance level. The drift share of 83.3% substantially exceeded the 30% auto-retrain threshold, confirming that the drift detection system responds correctly to significant distributional changes.

Drift Metric	Value
Total features monitored	12
Features detected as drifted	10 (83.3%)
Drift threshold	30%

Retrain triggered	Yes
Retrained model ROC-AUC	0.9970
Retrained model F1	0.7009

The retrained model showed a small reduction in F1 (0.7009 vs. 0.7945) on the original test set. This is expected: the retrained model was optimized on the combined distribution (original + drifted data) and would be expected to perform better on the new distribution while slightly compromising on the original test set, a fundamental trade-off in continual learning.

7.3 API Testing Results

All API endpoints were tested in Notebook 03 and via the automated pytest test suite. The following results were observed:

- **Normal transaction** (amount: 500, balanced accounts) - is_fraud: false, fraud_probability: 0.0000, risk_level: LOW.
- **Suspicious transaction** (amount: 180,000, balance wiped to zero, TRANSFER type) - is_fraud: true, fraud_probability: 0.9986, risk_level: HIGH.
- **Batch comparison of 10 randomly sampled test-set transactions via the API vs. direct model inference** - 0/10 mismatches (100% consistency).
- All 9 pytest tests passed in CI, including endpoint availability, response schema validation, input validation (422 on missing fields), and authentication (401 on wrong API key for /retrain).

7.4 Live Deployment Results

The application was successfully deployed to Railway cloud with a public URL. The deployment was verified with the following health check response:

```
GET /health → 200 OK
{"status": "healthy", "model": "best_model.pkl", "features": 12, "timestamp":
"..."}
```

The live monitoring dashboard was confirmed working, displaying real-time prediction counts, fraud detection statistics, recent prediction history table, drift monitoring status, and system information. The GitHub Actions CI/CD pipeline completed successfully across all 11 recorded pipeline runs.

7.5 Discussion

The results demonstrate that the MLOps platform successfully fulfils all stated objectives. The XGBoost model achieves production-quality fraud detection performance, the API responds correctly under both legitimate and fraudulent inputs, and the drift monitoring system correctly identifies distributional shifts and triggers automated retraining.

One notable limitation is that the drift detection feature_drift field in the JSON summary shows all features as false despite the 10 detected drifted features. This is a minor parsing artefact caused by a key name difference in Evidently 0.7.21 (the per-feature results require iteration through all metrics objects rather than a single dictionary key). The dashboard and overall drift share calculation are unaffected by this issue, the retrain trigger and report generation work correctly.

8. Challenges Faced

8.1 Evidently API Version Compatibility

The most time-consuming technical challenge was an API breaking change in Evidently AI. Evidently released a major redesign in version 0.5+ that completely changed the import paths and class names. The notebook initially used the 0.4.x API (evidently.report.Report, evidently.metric_preset.DataDriftPreset), which caused ModuleNotFoundError on first run. The installed version was 0.7.21 (the new API). Resolution required identifying the correct import structure and adapting the metric result parsing. Specifically, the per-feature drift results are stored in separate metric objects rather than a single drift_by_columns dictionary, requiring iteration through all 18 metrics objects to extract column-level results.

8.2 Dependency Conflicts and Docker Build Failures

The initial Docker build failed because requirements.txt included python-multipart = 1.3.1, a version that does not exist on PyPI. The pip freeze approach to generating requirements files can capture packages that are installed from non-standard sources or with version specifiers that do not resolve correctly in a clean environment. Resolution required manually identifying and removing the invalid entry, then verifying the remaining dependencies in the Docker build environment.

8.3 Class Imbalance and Metric Selection

The heavily imbalanced nature of the dataset (0.13% fraud rate) initially produced misleading results: the logistic regression baseline appeared to perform well by accuracy (98.2%) while being nearly useless in practice. Selecting appropriate evaluation metrics (F1 score, precision, recall, ROC-AUC) and understanding the trade-offs between them was critical to correctly interpreting model performance and selecting XGBoost as the production model.

8.4 Docker Desktop Installation and Port Conflicts

Docker Desktop was not initially installed on the development machine (Windows), requiring a fresh installation including WSL2 setup. During testing, port conflicts arose when docker-compose up and uvicorn were run simultaneously on port 8000. Resolution required terminating one service before starting the other, and understanding the distinction between the development (uvicorn --reload) and production (Docker) workflows.

8.5 Render.com Free Tier Card Requirement

The initially planned deployment target (Render.com) began requiring credit card verification even for free-tier accounts. As an internship project without billing information, this deployment path was blocked. Railway.app was adopted as an alternative, which required updating the deployment configuration from render.yaml to railway.json and adjusting the Dockerfile CMD to support Railway's dynamic PORT environment variable.

8.6 MLflow on Windows — Process Spawning Issues

Running the MLflow UI on Windows produced recurring child process crashes with OSError: [WinError 10022]. This is a known issue with MLflow's multiprocessing architecture on Windows, where worker processes fail to bind to pre-allocated sockets. The system auto-recovers by respawning workers, but the terminal output is noisy. The MLflow UI remained functional despite these messages, correctly serving experiment data at http://localhost:5000.

8.7 Drift Simulation Threshold Calibration

The first drift simulation (40% amount increase) was insufficient to cross Evidently's statistical significance threshold, resulting in only 2 of 13 features (15.4%) being detected as drifted — below the 30% trigger threshold. A more aggressive simulation was required (3× amount increase, 2.5× balance scaling, 400-step time shift) to produce a meaningful demonstration of the auto-retrain capability. This highlights a real-world challenge: production drift may be gradual, and thresholds must be calibrated carefully to avoid both false alarms and missed drift events.

9. Conclusion

This project was able to develop and deploy an entire and production-level MLOps system for real-time fraud detection in finance in two weeks as an intern. All eight deliverables were achieved as follows: a complete GitHub repository, end-to-end ML pipeline developed using four Jupyter notebooks, containerized application, live deployment of the system in Railway with a working URL, GitHub Actions for continuous integration & delivery (CI/CD) along with testing, live dashboard with real-time statistics, README file containing full technical documentation, and architecture diagram.

The results obtained from the training of the XGBoost classifier based on the PaySim data were excellent with regards to the accuracy of fraud detection: ROC-AUC of 0.9960 and F1 score of 0.7945 for the test set. It should be mentioned that the improvement in the F1 score was 315%, compared to the logistic regression benchmark model, while having identical levels of recall.

Besides technical excellence, this project shows the entire life cycle of developing an enterprise AI system starting from data acquisition up to the deployment and monitoring of the entire production system in a version-controlled, containerized, continuously deployed, and continuously monitored environment. The technologies and architecture used in this project, including MLflow experiment tracking, FastAPI model serving, Docker containerization, Evidently drift detection, and GitHub Actions CI/CD, are the best practices in the industry.

10. Future Improvements

- **Real-time streaming pipeline**: integrate Apache Kafka to enable the system to ingest live transaction streams rather than batch files, enabling true real-time fraud scoring at scale.
- **Advanced model architectures**: evaluate deep learning approaches (neural network ensembles) and AutoML frameworks to potentially improve F1 further.
- **Model explainability**: integrate SHAP to provide per-prediction feature attribution, enabling fraud analysts to understand why a specific transaction was flagged.
- **Scheduled drift monitoring**: implement APScheduler within the FastAPI application to automatically run drift checks every 6 hours against a rolling window of recent predictions, replacing the manual /retrain endpoint trigger.
- **Persistent monitoring store**: migrate MLflow from local filesystem (mlruns/) to a PostgreSQL database backend to enable persistent storage across container restarts on Railway.
- **Alert system**: integrate email or Slack notifications when drift is detected or when model performance metrics drop below defined thresholds.
- **Authentication and rate limiting**: add OAuth2 or API key authentication to the /predict endpoint to secure the production API, alongside rate limiting to prevent abuse.
- **Extended test coverage**: increase pytest coverage to include drift monitoring functions, the retrain workflow, and edge cases such as extreme feature values and concurrent requests.
- **Cloud-native MLflow**: migrate to a managed MLflow instance to enable team collaboration and persistent experiment history across deployments.

11. References

[1] Anthropic. (2024). Evidently AI Documentation. https://docs.evidentlyai.com/

[2] Docker Inc. (2024). Docker Documentation. https://docs.docker.com/

[3] FastAPI Documentation. (2024). https://fastapi.tiangolo.com/

[4] GitHub Actions Documentation. (2024). https://docs.github.com/en/actions

[5] MLflow Documentation. (2024). https://mlflow.org/docs/latest/index.html

[6] Railway Documentation. (2024). https://docs.railway.app/

12. Appendix

Appendix A: API Request / Response Example

The following shows a complete interaction with the /predict endpoint for a high-risk transaction:

```
POST /predict HTTP/1.1
Host: fraud-detection-mlops-platform-production-production.up.railway.app
Content-Type: application/json
```

```
{
  "step": 1,
  "amount": 180000.0,
  "oldbalanceOrg": 180000.0,
  "newbalanceOrig": 0.0,
  "oldbalanceDest": 0.0,
  "newbalanceDest": 0.0,
  "type_encoded": 1,
  "orig_balance_diff": 180000.0,
  "dest_balance_diff": 0.0,
  "orig_balance_zero": 1,
  "amount_to_balance_ratio": 1.0,
  "hour_of_day": 1
}
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "is_fraud": true,
  "fraud_probability": 0.9986,
  "risk_level": "HIGH",
  "prediction_time": "2026-05-31T13:00:00+00:00"
}
```


Appendix B: Feature Engineering Reference

Feature Name	Formula / Source	Type	Rationale
step	Raw column	Integer	Time step of simulation
amount	Raw column	Float	Transaction amount
oldbalanceOrg	Raw column	Float	Sender pre-transaction balance
newbalanceOrig	Raw column	Float	Sender post-transaction balance
oldbalanceDest	Raw column	Float	Recipient pre-transaction balance
newbalanceDest	Raw column	Float	Recipient post-transaction balance
type_encoded	1 if TRANSFER else 0	Binary	Fraud-relevant transaction type indicator
orig_balance_diff	oldbalanceOrg - newbalanceOrig	Float	Expected to equal amount; discrepancy signals fraud
dest_balance_diff	newbalanceDest - oldbalanceDest	Float	Often zero in fraud (money moved elsewhere)
orig_balance_zero	1 if newbalanceOrig == 0 else 0	Binary	Account-draining indicator — strong fraud signal
amount_to_balance_ratio	amount / (oldbalanceOrg + 1e-9), clipped at 10	Float	Proportion of balance transferred
hour_of_day	step % 24	Integer	Hour of day — captures diurnal patterns

Appendix C: GitHub Repository Contents

Repository URL: https://github.com/nisansalasandu/fraud-detection-mlops-platform

Live API URL: https://fraud-detection-mlops-platform-production-production.up.railway.app

Live Dashboard: https://fraud-detection-mlops-platform-production-production.up.railway.app/dashboard

Appendix D: Drift Monitoring Summary (from monitoring/reports/latest_drift_summary.json)


```
{
  "timestamp": "2026-05-29T12:49:33.163212",
  "dataset_drifted": true,
  "drifted_features": 10,
  "total_features": 12,
  "drift_share": 0.8333,
  "threshold": 0.3,
  "retrain_triggered": true
}
```